```
import streamlit as st
import yfinance as yf
import numpy as np
import pandas as pd
import requests
from datetime import datetime, timezone
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, Vo
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from textblob import TextBlob
from supabase import create_client

SUPABASE_URL = "https://uwhfboxiuvkorhhjeypy.supabase.co"
SUPABASE_KEY = "YOUR_ANON_KEY_HERE"

supabase = create_client(SUPABASE_URL, SUPABASE_KEY)

st.set_page_config(layout="wide")
st.title("Crypto Prediction Dashboard")

total_balance = 18.97
min_allocation = 12.0
max_per_trade = 0.20
max_leverage = 5

cryptos = ["BTC-USD", "ETH-USD", "ADA-USD", "XRP-USD", "SOL-USD", "DOGE-USD", "LI

crypto_names = {
    "BTC-USD": "bitcoin",
    "ETH-USD": "ethereum",
    "ADA-USD": "cardano",
    "XRP-USD": "ripple",
    "SOL-USD": "solana",
    "DOGE-USD": "dogecoin",
    "LINK-USD": "chainlink",
    "LTC-USD": "litecoin",
    "BCH-USD": "bitcoin-cash",
    "ZEN-USD": "horizen"
}

pairs = list(zip(cryptos[::2], cryptos[1::2]))

def save_prediction(crypto, signal, prediction, current_price):
    try:
```

```
        supabase.table("predictions").insert({
            "crypto": crypto,
            "signal": signal,
            "prediction": float(prediction),
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "was_correct": None
        }).execute()
    except Exception:
        pass

def update_past_predictions():
    try:
        result = supabase.table("predictions").select("*").is_("was_correct", "nu
        for row in result.data:
            try:
                ticker = yf.Ticker(row["crypto"])
                data = ticker.history(period="2d", interval="1h")
                if len(data) > 1:
                    actual_return = float(data["Close"].pct_change().iloc[-1])
                    was_correct = (row["signal"] == "LONG" and actual_return > 0)
                    supabase.table("predictions").update({
                        "was_correct": was_correct,
                        "actual_return": actual_return
                    }).eq("id", row["id"]).execute()
            except Exception:
                pass
    except Exception:
        pass

def get_historical_accuracy(crypto):
    try:
        result = supabase.table("predictions").select("*").eq("crypto", crypto).n
        if result.data and len(result.data) > 0:
            correct = sum(1 for r in result.data if r["was_correct"])
            return correct / len(result.data)
        return None
    except Exception:
        return None

def calculate_rsi(prices, period=14):
    delta = prices.diff()
    gain = delta.where(delta > 0, 0).rolling(window=period).mean()
    loss = -delta.where(delta < 0, 0).rolling(window=period).mean()
    rs = gain / loss
    return 100 - (100 / (1 + rs))

def get_news_sentiment(crypto_name):
```

```
    try:
        url = "https://api.coingecko.com/api/v3/news"
        response = requests.get(url, timeout=5)
        if response.status_code == 200:
            articles = response.json().get("data", [])
            sentiments = []
            for article in articles:
                title = article.get("title", "")
                description = article.get("description", "")
                text = title + " " + description
                if crypto_name.lower() in text.lower():
                    sentiment = TextBlob(text).sentiment.polarity
                    sentiments.append(sentiment)
            if sentiments:
                return sum(sentiments) / len(sentiments)
        return 0.0
    except Exception:
        return 0.0

def get_fear_greed():
    try:
        url = "https://api.alternative.me/fng/"
        response = requests.get(url, timeout=5)
        if response.status_code == 200:
            value = int(response.json()["data"][0]["value"])
            return (value - 50) / 50
        return 0.0
    except Exception:
        return 0.0

def calculate_features(data):
    data["returns"] = data["Close"].pct_change().fillna(0)
    data["ma_5"] = data["Close"].rolling(5).mean()
    data["ma_20"] = data["Close"].rolling(20).mean()
    data["ma_50"] = data["Close"].rolling(50).mean()
    data["ma_200"] = data["Close"].rolling(200).mean()
    data["rsi"] = calculate_rsi(data["Close"])
    data["macd"] = data["Close"].ewm(span=12).mean() - data["Close"].ewm(span=26)
    data["macd_signal"] = data["macd"].ewm(span=9).mean()
    data["bollinger_upper"] = data["Close"].rolling(20).mean() + 2 * data["Close"
    data["bollinger_lower"] = data["Close"].rolling(20).mean() - 2 * data["Close"
    data["bollinger_width"] = data["bollinger_upper"] - data["bollinger_lower"]
    data["volume_ma"] = data["Volume"].rolling(10).mean()
    data["volume_ratio"] = data["Volume"] / data["volume_ma"]
    data["price_momentum"] = data["Close"].pct_change(5)
    data["volatility"] = data["returns"].rolling(10).std()
    data["high_low_range"] = (data["High"] - data["Low"]) / data["Close"]
```

```
    data["close_position"] = (data["Close"] - data["Low"]) / (data["High"] - data
    return data.dropna()

@st.cache_data(ttl=3600)
def get_signal(crypto):
    coin_name = crypto_names[crypto]
    ticker = yf.Ticker(crypto)
    train_data = ticker.history(period="90d", interval="1d")
    current_price = float(train_data["Close"].iloc[-1])

    train_data = calculate_features(train_data)

    news_sentiment = get_news_sentiment(coin_name)
    fear_greed = get_fear_greed()

    feature_cols = [
        "ma_5", "ma_20", "ma_50",
        "rsi", "macd", "macd_signal",
        "bollinger_upper", "bollinger_lower", "bollinger_width",
        "Volume", "volume_ma", "volume_ratio",
        "price_momentum", "volatility",
        "high_low_range", "close_position"
    ]

    X = train_data[feature_cols].values.astype(float)
    y = train_data["returns"].values.astype(float)

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.

    rf = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42)
    gb = GradientBoostingRegressor(n_estimators=100, max_depth=5, learning_rate=0
    ensemble = VotingRegressor([("rf", rf), ("gb", gb)])
    ensemble.fit(X_train, y_train)
    accuracy = ensemble.score(X_test, y_test)

    latest = scaler.transform([train_data[feature_cols].iloc[-1].values.astype(fl
    prediction = ensemble.predict(latest)[0]

    combined_score = prediction + (news_sentiment * 0.3) + (fear_greed * 0.2)

    signal = "LONG" if combined_score > 0 else "SHORT"
    take_profit = current_price * 1.05
    stop_loss = current_price * 0.97
```

```
    confidence = min(abs(combined_score) * 100, 100)
    if confidence > 66:
        confidence_label = "High Confidence"
        allocation_pct = max_per_trade
        suggested_leverage = min(max_leverage, 5)
    elif confidence > 33:
        confidence_label = "Medium Confidence"
        allocation_pct = max_per_trade * 0.5
        suggested_leverage = min(max_leverage, 3)
    else:
        confidence_label = "Low Confidence"
        allocation_pct = max_per_trade * 0.25
        suggested_leverage = 1

    suggested_amount = round(total_balance * allocation_pct, 2)
    suggested_amount = max(suggested_amount, min_allocation)
    can_afford = total_balance >= suggested_amount

    rsi_val = float(train_data["rsi"].iloc[-1])
    if rsi_val > 70:
        rsi_note = "Overbought"
        close_alert = True
    elif rsi_val < 30:
        rsi_note = "Oversold"
        close_alert = True
    else:
        rsi_note = "Neutral"
        close_alert = False

    ma5 = float(train_data["ma_5"].iloc[-1])
    ma20 = float(train_data["ma_20"].iloc[-1])
    if signal == "LONG" and ma5 < ma20:
        close_alert = True
    elif signal == "SHORT" and ma5 > ma20:
        close_alert = True

    sentiment_label = "Positive" if news_sentiment > 0 else "Negative" if news_se
    fg_label = "Greed" if fear_greed > 0 else "Fear"

    save_prediction(crypto, signal, prediction, current_price)
    historical_accuracy = get_historical_accuracy(crypto)

    return signal, suggested_amount, suggested_leverage, current_price, take_prof

update_past_predictions()

all_signals = []
```

```
for crypto in cryptos:
    try:
        result = get_signal(crypto)
        all_signals.append((crypto, result))
    except Exception:
        pass

all_signals.sort(key=lambda x: abs(x[1][14]), reverse=True)

affordable = [(c, r) for c, r in all_signals if r[15]]
top_3 = affordable[:3]

st.markdown("---")
st.subheader("Best Trades Right Now")
if top_3:
    for crypto, result in top_3:
        signal, suggested_amount, suggested_leverage, current_price, take_profit,
        color = "green" if signal == "LONG" else "red"
        st.markdown("**" + crypto + "** - :" + color + "[" + signal + "] | Invest
else:
    st.warning("Balance of $" + str(total_balance) + " is below the $" + str(min_

st.markdown("---")
st.subheader("All Crypto Signals")

close_alerts = []

for left, right in pairs:
    col1, col2 = st.columns(2)
    for col, crypto in zip([col1, col2], [left, right]):
        with col:
            st.markdown("### " + crypto)
            try:
                signal, suggested_amount, suggested_leverage, current_price, take
                color = "green" if signal == "LONG" else "red"
                st.markdown("**Signal:** :" + color + "[" + signal + "] | " + con
                st.write("Entry: $" + str(round(current_price, 2)) + " | TP: $" +
                if can_afford:
                    st.write("Invest: $" + str(suggested_amount) + " at " + str(s
                else:
                    st.error("Need $" + str(suggested_amount) + " minimum")
                st.write("RSI: " + str(round(rsi_val, 2)) + " (" + rsi_note + ")"
                st.write("News: " + sentiment_label + " | Market: " + fg_label)
                st.write("Accuracy: " + str(round(accuracy * 100, 2)) + "%")
                if historical_accuracy is not None:
                    st.write("Win Rate: " + str(round(historical_accuracy * 100,
                if close_alert:
```

```
                    st.warning("CLOSE NOW!")
                    close_alerts.append(crypto)
            except Exception as e:
                st.write("Error: " + str(e))
            st.divider()

if close_alerts:
    st.error("CLOSE ALERTS: " + ", ".join(close_alerts))
```